GUIDE PROCEDURAL

Deploiement de sites web SEO

via Intelligence Artificielle (Claude)

Acquisition de leads par SEO local ultra longue traine

Sites HTML statiques | Formulaires Netlify | Maillage interne

Version 1.0 — Mars 2026

1. Introduction

Ce guide formalise le processus complet de creation et de deploiement de sites web statiques optimises pour le SEO local, en utilisant Claude (Anthropic) comme assistant central de production. L'objectif est l'acquisition de leads qualifies via des requetes ultra longue traine geolocalises.

Cas d'usage type : creation d'un site vitrine pour un prestataire de services (ex. : eradication de punaises de lit) avec une page dediee par ville d'un departement, chaque page ciblant des requetes SEO locales specifiques.

Philosophie du workflow : separer la reflexion strategique (Claude Web) de l'execution technique (Claude Code via VS Code), puis deployer rapidement via un pipeline GitHub > Netlify.

2. Ecosysteme technique : les 4 plateformes

Le workflow repose sur quatre outils complementaires, chacun intervenant a un moment precis du processus.

PLATEFORME	UTILITE PRINCIPALE	MOMENT D'INTERVENTION	VALEUR AJOUTEE
Claude Web	Strategie, brief, recherche de mots-cles, redaction du cahier des charges	Phase 1 : Conception et briefing	Reflexion iterative, conversation longue, peaufinage du brief
VS Code + Claude Code	Generation du code HTML/CSS/JS, creation de toutes les pages, maillage interne	Phase 2 : Production technique	Execution autonome, generation en masse, memoire projet
GitHub	Hebergement du code source, versioning, depot distant (remote)	Phase 3 : Stockage et versioning	Historique des modifications, collaboration, rollback possible
Netlify	Deploiement automatique, DNS, formulaires, redirections, HTTPS	Phase 4 : Mise en ligne et exploitation	Deploiement en 1 clic, gestion formulaires native, CDN performant

3. Guide detaille par etapes

PHASE 1 — CONCEPTION ET BRIEFING (Claude Web)

Definir le projet et le brief

Logique	Poser les fondations strategiques : secteur, zone geographique, objectif de leads, type de prestation.

Action	Ouvrir une nouvelle conversation sur Claude.ai (web). Decrire le projet en langage naturel avec tous les elements business.
Outil	Claude Web (claude.ai)

PROMPT SUGGERE

Je souhaite creer un site web statique HTML pour une entreprise de [SECTEUR] (ex : eradication de punaises de lit) dans le departement [XX]. L'objectif est l'acquisition de leads via le SEO local ultra longue traine. Chaque ville principale du departement aura sa propre page. Le site doit etre ultra rapide, avec une structure semantique propre, du balisage Schema.org (JSON-LD LocalBusiness), des formulaires de contact Netlify Forms, un maillage interne solide et des boutons d'appel a l'action "Devis gratuit". Aide-moi a structurer le brief complet du projet.

Generer la liste des villes

Logique	Obtenir la liste exhaustive des villes principales du departement cible pour definir le perimetre de pages a creer.
Action	Demander a Claude de generer un fichier JSON structure avec toutes les villes, incluant les donnees utiles au SEO local.
Outil	Claude Web (claude.ai)

PROMPT SUGGERE

Genere-moi un fichier JSON contenant toutes les villes principales du departement [XX - Nom]. Pour chaque ville, inclus : nom, code postal, population estimee, slug URL (ex : nom-de-la-ville), et les quartiers ou zones notables. Ce fichier servira de base de donnees pour la generation automatique des pages.

Recherche de mots-cles et brief SEO

Logique	Definir les mots-cles principaux, les variantes longue traine par ville, et la strategie de contenu semantique.
Action	Travailler iterativement avec Claude Web pour affiner les mots-cles et la structure du contenu type par page.
Outil	Claude Web (claude.ai)

PROMPT SUGGERE

A partir de la thematique [SECTEUR] et de la liste de villes, propose-moi : 1) Les mots-cles principaux et secondaires. 2) Un template de structure de page type (H1, H2, H3) optimise SEO. 3) Les variations longue traine par ville. 4) Le balisage JSON-LD LocalBusiness adapte. 5) La strategie de maillage interne entre les pages de villes. Pense aussi aux Titles (mot-cle principal, pas de separateur) et aux meta descriptions optimisees pour le CTR.

Rediger le fichier de contexte (Markdown)

Logique	Compiler toutes les decisions strategiques dans un document unique qui servira de memoire de projet pour Claude Code.
Action	Creer un fichier .md (Markdown) synthetisant : le brief, les mots-cles, la structure des pages, les regles techniques, les consignes de style.
Outil	Claude Web (claude.ai)

PROMPT SUGGERE

A partir de tout ce que nous avons defini, redige un fichier Markdown complet de contexte projet. Ce fichier doit contenir : le brief general, la liste des mots-cles, le template de page, les regles de balisage (Hn, JSON-LD, canonicals), les consignes pour les formulaires Netlify Forms, les regles de nommage des fichiers, les consignes de maillage interne, et les points de vigilance (accents, tirets, majuscules dans les Hn). Ce fichier sera donne a Claude Code comme reference.

PHASE 2 — PRODUCTION TECHNIQUE (VS Code + Claude Code)

Configurer l'environnement VS Code

Logique	Preparer l'espace de travail local : dossier projet, arborescence, lien avec Claude Code.
Action	Creer un dossier dedie sur le disque dur (un dossier par site). Ouvrir ce dossier dans VS Code. Installer l'extension Claude Code. Relier Claude a VS Code via la fenetre de discussion integree.
Outil	Visual Studio Code + Claude Code

PROMPT SUGGERE

N/A — Action manuelle : 1) Creer le dossier du projet sur le disque dur. 2) Ouvrir VS Code > File > Open Folder. 3) Ouvrir la fenetre Claude Code (terminal ou panneau lateral). 4) Creer une memoire projet dans Claude Code pour ce site specifique.

Injecter le brief et lancer la generation

Logique	Transmettre le fichier de contexte Markdown a Claude Code pour qu'il ait toute la vision du projet avant de coder.
Action	Coller le contenu du fichier .md dans Claude Code, puis donner l'instruction de generer le site complet.
Outil	Claude Code (dans VS Code)

PROMPT SUGGERE

Voici le brief complet du projet [coller le fichier MD]. Avant de commencer, propose-moi un plan de production detaille : arborescence des fichiers, ordre de creation, et template de

page. Je veux approuver ce plan une seule fois, puis tu travailleras en autonomie pour generer toutes les pages sans validation intermediaire. Assure-toi d'integrer : les formulaires Netlify Forms (avec attribut netlify et name unique par formulaire), le JSON-LD LocalBusiness par ville, le maillage interne, les boutons "Devis gratuit", le dossier assets/ pour les images.

Approuver le plan et laisser Claude en autonomie

Logique	Valider le plan de production propose pour eviter les allers-retours. Claude Code generera ensuite toutes les pages en continu.
Action	Relire le plan propose par Claude Code. Si OK, valider et laisser Claude produire toutes les pages en mode autonome.
Outil	Claude Code (dans VS Code)

PROMPT SUGGERE

Le plan est approuve. Lance la production de toutes les pages en suivant exactement le plan valide. Ne me demande pas de validation a chaque page. Genere tout d'une traite. Previens-moi quand c'est fini.

Generer les fichiers techniques SEO

Logique	Completer le site avec les fichiers techniques indispensables au referencement et au bon fonctionnement sur Netlify.
Action	Demander a Claude Code de generer les fichiers complementaires : sitemap.xml, sitemap.html, fichier de redirections Netlify, robots.txt.
Outil	Claude Code (dans VS Code)

PROMPT SUGGERE

Maintenant, genere les fichiers techniques suivants : 1) sitemap.xml avec toutes les URLs du site. 2) sitemap.html (plan du site visible pour l'utilisateur avec liens vers toutes les pages). 3) Un fichier _redirects pour Netlify qui redirige toutes les URLs .html vers des URLs propres sans extension. 4) Les redirections HTTP vers HTTPS. 5) Un fichier robots.txt. 6) Verifie que chaque page a sa balise canonical correcte.

Optimiser les Title et Meta descriptions

Logique	Peaufiner les balises Title (mot-cle principal, sans separateur type | ou -) et les meta descriptions pour maximiser le CTR.
Action	Demander a Claude Code de revoir et optimiser chaque Title et meta description du site.
Outil	Claude Code (dans VS Code)

PROMPT SUGGERE

Passe en revue toutes les pages du site et optimise : 1) Les balises Title : le mot-cle principal doit etre en debut de title, pas de separateur (ni pipe ni tiret). 2) Les meta descriptions : redigees pour maximiser le taux de clic, avec appel a l'action et geolocalisation. 3) Verifie la coherence des balises H1 avec les Titles. Liste-moi toutes les modifications effectuees.

Ajouter les images (dossier assets/)

Logique	Integrer les visuels du site dans le dossier assets/ au format WebP pour la performance.
Action	Generer ou ajouter manuellement les images au format WebP dans le dossier assets/images/. Optimiser le poids.
Outil	Visual Studio Code (manuellement)

PROMPT SUGGERE

N/A — Action manuelle : 1) Creer le dossier assets/images/ dans le projet. 2) Generer les images (manuellement, via outil IA, ou banque d'images). 3) Convertir en WebP. 4) Nommer avec des noms descriptifs SEO (ex : eradication-punaises-de-lit-marseille.webp). 5) Verifier que les balises <img> dans le code ont des attributs alt optimises.

PHASE 3 — MISE EN LIGNE (GitHub + Netlify)

Pousser le site sur GitHub

Logique	Versionner le code et le rendre accessible a Netlify pour le deploiement automatique.
Action	Depuis VS Code ou le terminal Claude Code, initialiser le depot Git, commit, puis push vers GitHub.
Outil	VS Code + GitHub

PROMPT SUGGERE

C'est fini. Push le site complet sur GitHub. Initialise le depot git si ce n'est pas deja fait, fais un commit avec le message "Deploiement initial - site [NOM]", et push sur le depot distant [URL_GITHUB].

Connecter Netlify a GitHub

Logique	Automatiser le deploiement : a chaque push sur GitHub, Netlify reconstruit et publie le site.
Action	Sur netlify.com, creer un nouveau site en le reliant au depot GitHub du projet.
Outil	Netlify (netlify.com)

PROMPT SUGGERE

N/A — Action manuelle sur Netlify : 1) Se connecter a netlify.com. 2) "Add new site" > "Import an existing project" > GitHub. 3) Selectionner le depot du projet. 4) Configurer le build : laisser vide (site statique). 5) Deployer.

Verifier le site en ligne

Logique	Controle qualite complet : affichage desktop/mobile, formulaires, liens internes, performance.
Action	Ouvrir le site deploye par Netlify. Tester sur desktop et mobile. Verifier les formulaires, les redirections, le maillage interne.
Outil	Navigateur web + VS Code (pour corrections)

PROMPT SUGGERE

N/A — Checklist de verification : 1) Affichage correct desktop et mobile. 2) Formulaires Netlify : tester l'envoi, verifier dans le dashboard Netlify > Forms. 3) Redirections fonctionnelles (URLs sans .html). 4) HTTPS actif. 5) Liens internes et maillage. 6) Boutons "Devis" fonctionnels. 7) JSON-LD valide (tester via le Rich Results Test de Google). 8) Vitesse (PageSpeed Insights).

PHASE 4 — INDEXATION ET EXPLOITATION

Soumettre a Google Search Console

Logique	Declarer le site aupres de Google pour accelerer l'indexation des pages.
Action	Connecter le site a Google Search Console via le compte Gmail. Soumettre le sitemap.xml.
Outil	Google Search Console

PROMPT SUGGERE

N/A — Action manuelle : 1) Aller sur search.google.com/search-console. 2) Ajouter la propriete (URL du site Netlify ou domaine personnalise). 3) Verifier la propriete (methode DNS ou fichier HTML). 4) Soumettre le sitemap : Sitemaps > Ajouter > sitemap.xml.

Creer le guide / FAQ / Centre d'aide

Logique	Enrichir le site avec du contenu complementaire pour renforcer l'autorite semantique et capter des requetes informationnelles.
Action	Utiliser Claude Web pour rediger un guide, une FAQ et un centre d'aide lies a la thematique.
Outil	Claude Web + Claude Code

PROMPT SUGGERE

Redige un guide complet sur [THEMATIQUE] (ex : "Guide complet : comment se debarrasser des punaises de lit") avec une FAQ de 15-20 questions courantes. Le guide doit etre optimise SEO avec une structure Hn propre. Inclus un schema FAQ (JSON-LD FAQPage) pour le balisage. Ce contenu sera ajoute au site existant.

Monitoring et iterations

Logique	Suivre les performances, corriger, optimiser en continu.
Action	Utiliser Haloscan pour auditer, Search Console pour suivre l'indexation, et iterer via le cycle VS Code > GitHub > Netlify.
Outil	Tous les outils

PROMPT SUGGERE

N/A — Routine de suivi : 1) Search Console : suivre l'indexation, les requetes, les erreurs. 2) Haloscan : audit SEO technique (copies d'ecran). 3) Netlify Forms : recuperer les leads, configurer les tags pour la redistribution. 4) Pour toute modification : editer dans VS Code > commit > push > deploiement automatique Netlify.

4. Lexique des commandes essentielles

Commandes Git les plus utilisees dans ce workflow, a executer depuis le terminal de VS Code ou Claude Code.

COMMANDE	SYNTAXE	DESCRIPTION
Commit	`git commit -m "message"`	Enregistre les modifications localement avec un message descriptif
Push	`git push origin main`	Envoie les commits locaux vers GitHub (declenche le deploiement Netlify)
Add	`git add .`	Prepare tous les fichiers modifies pour le prochain commit
Status	`git status`	Affiche les fichiers modifies, ajoutes ou non suivis
Pull	`git pull origin main`	Recupere les dernieres modifications depuis GitHub
Init	`git init`	Initialise un nouveau depot Git dans le dossier courant
Clone	`git clone [URL]`	Telecharge un depot distant en local

5. Points de vigilance

Liste des erreurs frequentes et des pieges a eviter lors de l'utilisation de Claude pour la generation de sites web.

5.1 Patterns IA a surveiller

ATTENTION : Defauts recurrents de Claude

Claude a tendance a introduire des patterns reconnaissables qu'il faut systematiquement corriger. Verifier apres chaque generation : les accents manquants ou incorrects, les tirets cadratin utilises a la place de tirets simples, les majuscules abusives dans les titres Hn (seule la premiere lettre doit etre en majuscule en francais).

- **Accents :** Verifier que tous les accents sont presents et corrects (e, e, a, u, c). Claude oublie parfois des accents.
- **Tirets :** Remplacer les tirets cadratin (—) par des tirets normaux (-) dans les contenus web.
- **Majuscules Hn :** En francais, seul le premier mot d'un titre prend une majuscule. Corriger les titres type 'Nos Services De Desinsectisation'.
- **Formulations generiques :** Demander a Claude de varier les tournures et d'eviter les phrases types IA comme 'Dans un monde ou...' ou 'Il est important de noter que...'.

5.2 Erreurs techniques courantes

- **Formulaires Netlify :** Chaque formulaire doit avoir l'attribut netlify et un name unique. Sans cela, Netlify ne captera pas les soumissions.
- **Canonical :** Chaque page doit avoir une balise <link rel='canonical'> pointant vers sa propre URL canonique. Oubli frequent.
- **Redirections :** Le fichier _redirects doit etre a la racine du projet. Tester que les URLs sans .html fonctionnent bien.
- **HTTPS :** Configurer la redirection HTTP > HTTPS dans Netlify ET dans le fichier _redirects.
- **JSON-LD :** Valider le balisage avec le Rich Results Test de Google apres deploiement.
- **Images :** Toujours au format WebP, avec des noms descriptifs et des attributs alt remplis. Ne jamais laisser un alt vide.

5.3 Bonnes pratiques de workflow

- **Un dossier par site :** Creer un dossier dedie par projet pour eviter les melanges. Chaque site a son propre depot GitHub.
- **Memoire Claude Code :** Creer une memoire specifique a chaque projet dans Claude Code pour maintenir la coherence sur la duree.
- **Plan de validation :** Approuver le plan UNE fois au debut, puis laisser Claude en autonomie. Eviter les validations intermediaires qui cassent le rythme.
- **Commit reguliers :** Faire un commit a chaque etape significative avec un message clair. Facilite le rollback en cas de probleme.
- **Donnees CSV :** Pour les donnees brutes (villes, codes postaux, etc.), privilegier le format CSV comme source de donnees structurees.

5.4 Checklist de lancement

CHECKLIST AVANT MISE EN PRODUCTION

Verifier systematiquement chaque point avant d'envoyer le site au client ou de le declarer en production.

1. Toutes les pages de villes sont generees et accessibles
2. Les Titles sont optimises (mot-cle en debut, pas de separateur)
3. Les meta descriptions sont uniques et orientees CTR
4. Les canonicals pointent vers les bonnes URLs
5. Le maillage interne est coherent et complet
6. Les formulaires Netlify fonctionnent (test d'envoi effectue)
7. Le JSON-LD est valide sur chaque page
8. Le sitemap.xml est complet et soumis a Search Console
9. Les redirections fonctionnent (URLs propres, HTTPS)
10. Le site est responsive (teste sur mobile et desktop)
11. Les images sont en WebP avec alt remplis
12. La vitesse est acceptable (PageSpeed Insights > 90)
13. Les patterns IA sont corriges (accents, tirets, majuscules)